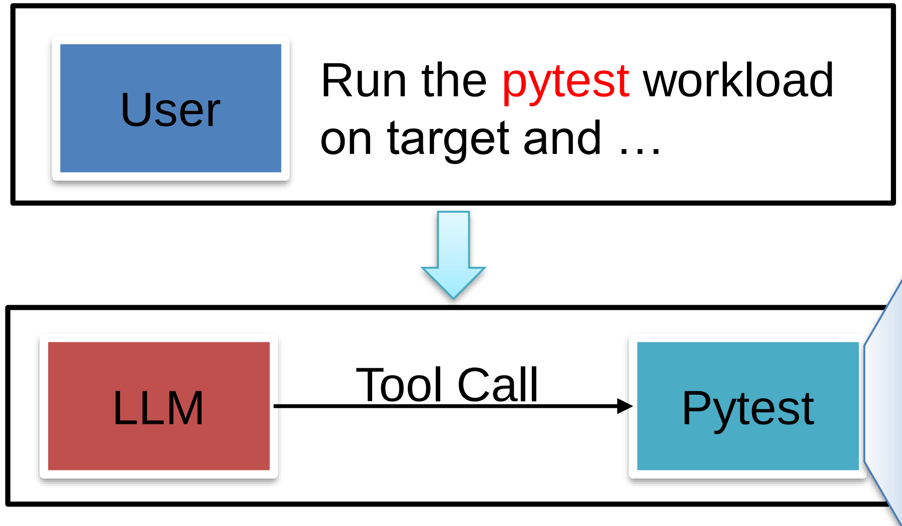
DBMS Final Project: ArtifactStore

2026/05/29

Outline

Motivation: Tool Outputs Overload Agent Context




```
========================== test session starts ======================
platform darwin -- Python 3.11.7, pytest-8.2.0, pluggy-1.5.0
rootdir: /Users/dev/example-app
collected 47 items

tests/test_login.py .......... [ 21%]
tests/test_refresh.py ........ [ 38%]
tests/test_auth.py .....F..... [ 61%]
tests/test_session.py ............ [ 87%]
tests/test_oauth.py ...... [100%]

========================== FAILURES ============================
________________________ test_auth_expiry __________________________

self = <tests.test_auth.TestAuthExpiry object at 0x10e1c2d50>

def test_auth_expiry(self):
token = mint_token(user_id=42, ttl_seconds=3600)
# advance the clock by 30 minutes; token should still be valid
with freeze_time(datetime.now() + timedelta(minutes=30)):
> assert validate_token(token) is True, "token expired prematurely"
…… more
```

Ex: Total token=9,609

- Tool Call Results往往非常龐大
- 佔據和擠壓到上下文視窗
- 因此tool call results不能直接塞回context window

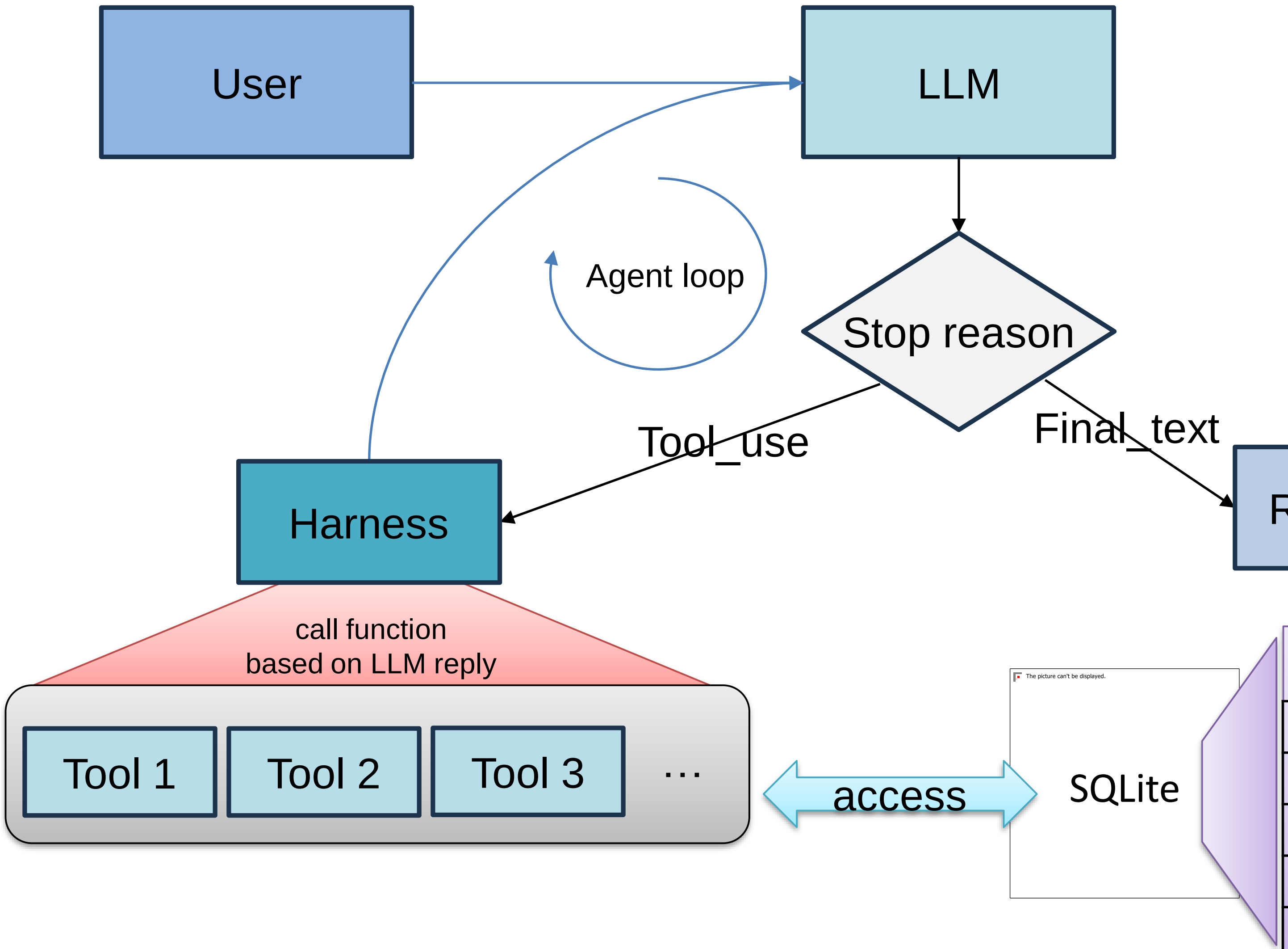
Existing Ways to Handle Large Tool Outputs

Approach	Description	Pros and Cons
Raw accumulation	keep tool outputs in context.	
Truncation / masking	remove or hide older observations.	
Summarization / compa	compress tool history into summaries.	
Retrieval / memory	store external information and retrieve relevant chunks	

引出我們的想法：Artifact Store

- Build an SQLite DB to store the important evidence
- Create A Sub-Agent and delegate it to solve the user prompt
- When delegating Sub-agent, we create a grant simultaneously to limit the scope and token budget that it can read and consume

Harness: Agent Loop



Artifact Store	
Artifact	每一筆tool results
Artifact Spans	從Tool Results分割出來的spans
Grants	門禁儲值卡資料
Access Logs	存取得的紀錄
Artifact Links	Artifact之間的關聯資料

Harness: Tool call

LLM side

decision maker

The model does not call Python functions directly. It emits structured `tool_use` requests such as:

```
tool_name: artifact_expand_view
input: {
  artifact_id: art_...,
  view: evidence
}
```

Jason schema

Tool Call

Harness side

policy enforcer

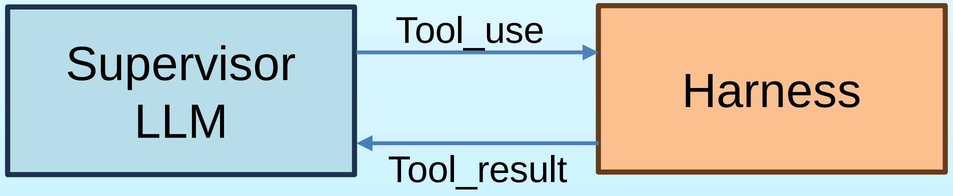
The local Python harness validates the request, executes the function, enforces grants, and returns a safe summary.

- **1. Check grant**
  operation, view, predicate, budget
- **2. Run function**
  query ArtifactStore / verify citation
- **3. Return safe result**
  no raw output unless explicitly granted

In both supervisor and subagent loops, the LLM proposes actions; the harness is the trusted execution boundary.

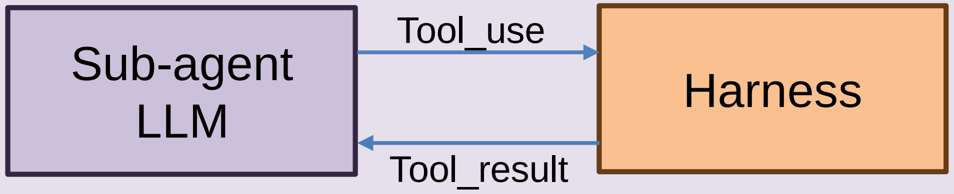
Agent Tools

Supervisor Agent



Tool Name	Function
1. Run_workload	執行工具並存入DB
2. Create_grant	建立門禁儲值卡
3. Delegate	委託子代理
4. Expand_artifact	展開某個VIEW
5. Verify_citation	驗證子代理回傳

Sub-Agent



Tool Name	Function
1. Artifact_search	搜尋DB
2. Artifact_get_spans	取得實際Spans
3. Artifact_expand_view	展開artifact_type
4. Artifact_find_related	搜尋關聯性
5. Submit_report	子代理回傳結果

Data Model

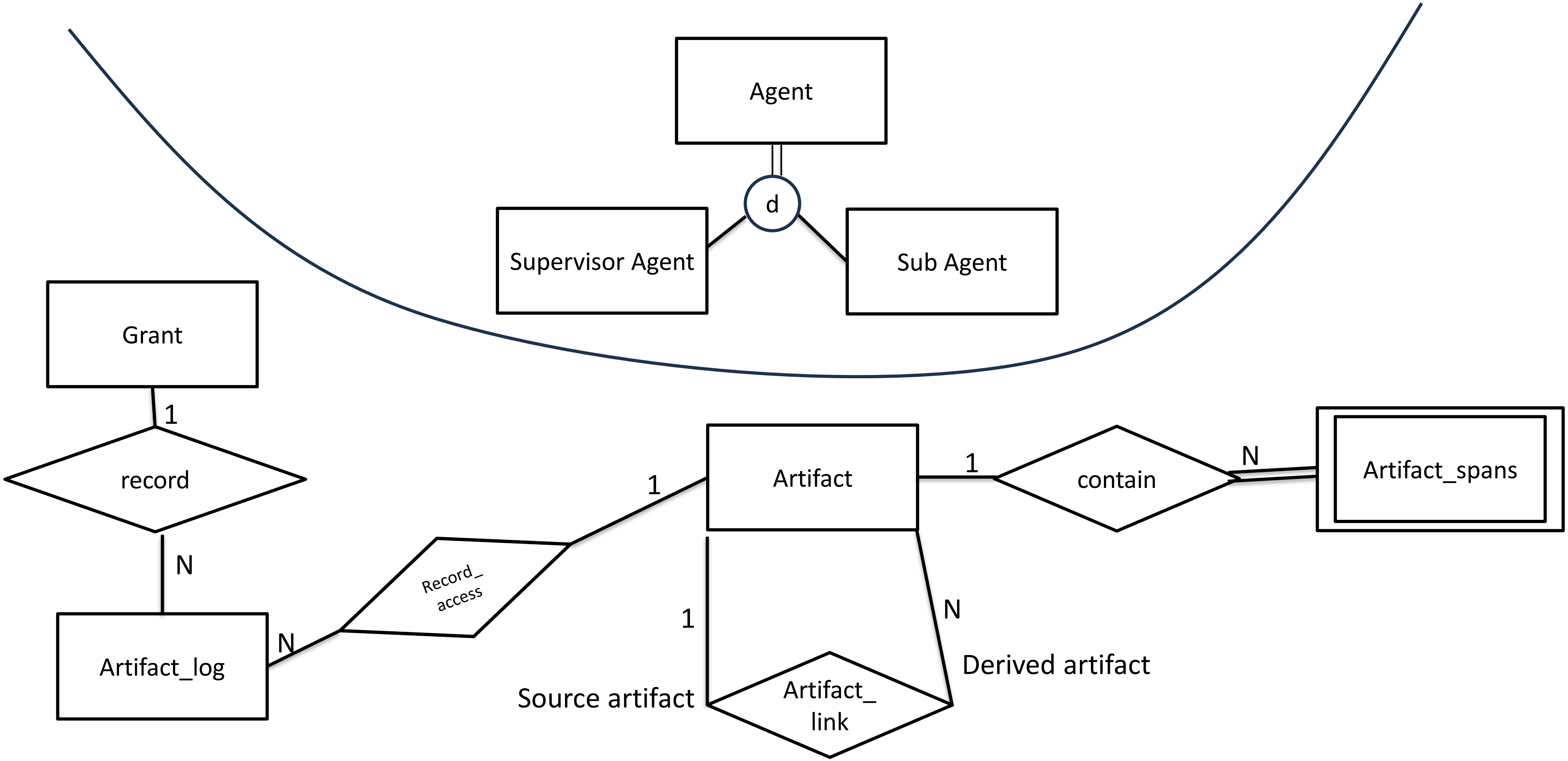

Agent
d
Supervisor Agent
Sub Agent
Grant
1
record
N
Artifact_log
N
Record_
access
1
Artifact
1
contain
N
Artifact_spans
1
N
Source artifact
Artifact_
link
Derived artifact

Table Schemas

Table	Purpose
artifacts	Core stored tool-result artifact.
artifact_spans	Citeable evidence spans extracted from artifacts.
artifact_links	Optional artifact-to-artifact relation/provenance edges.
artifact_grants	Capability-like scoped access record.
artifact_access_log	Audit log for all read attempts.
artifact_fts	FTS5 search index over previews and span text.

Column	Meaning
artifact_id	Artifact id returned to agents, e.g. art_xxxxxxxx.
session_id	Groups artifacts by run/session.
parent_artifact_id	Optional pointer for derived artifacts.
creator_agent_id	Agent or harness that created this artifact.
tool_name	Tool that produced the raw output.
artifact_type	Type-driven extractor key.
raw_uri	Reserved for future file-backed raw storage.
raw_blob	Canonical raw text payload stored in SQLite.
raw_hash	SHA-256 hex hash of raw text.
token_count	Estimated raw token count.
preview	Small safe preview of artifact.
sensitivity_label	Sensitivity class for predicate checks.
metadata_json	Artifact-type-specific metadata.
created_at	Insert timestamp.

Column	Meaning
span_id	Stable citeable span id, e.g. span_xxxxxxxx.
artifact_id	Parent artifact for this span.
span_type	Type of extracted evidence span.
file_path	Optional source file path associated with span.
line_start	Start line when available.
line_end	End line when available.
text	Span text returned to agents when authorized.
token_count	Estimated span token count.
importance	Extractor-assigned importance score.

Column	Meaning
artifact_id	Artifact id returned from FTS matches.
artifact_type	Artifact type searchable by FTS.
preview	Preview text searchable by FTS.
span_text	Joined extracted span text searchable by FTS.
tool_name	Tool name searchable by FTS.

- FTS5 是SQLite的Extension
- 他可以讓存入的資料支援Full-text search功能
- 其他的表格說明放在Appendix

Artifact API(Client-Side Program)

Slide 8：System WorkFlow

- 用方塊流程圖呈現整體系統架構

Slide 9：Experiments

- 展示最開頭提到的不同方法的數據結果(raw, truncated, summary, artifact store)
- Metrics是什麼：token 消耗、latency、問題成功率…
- 待討論：要展示單一agent嗎(B1-B4)，如果要的話那麼需要說明單一agent怎麼操作artifact store，或者可以展示雙agent時，sub agent看的回傳的比較(D1-D3)。
- 或者是我們再設計一下實驗，例如首先我們跑了單一agent，然後到雙agent。

Slide 10：目前的實驗數據

- 展示最開頭提到的不同方法的數據結果(raw, truncated, summary, artifact store)
- Metrics是什麼：token 消耗、latency、問題成功率…
- 待討論：要展示單一agent嗎(B1-B4)，如果要的話那麼需要說明單一agent怎麼操作artifact store，或者可以展示雙agent時，sub agent看的回傳的比較(D1-D3)。
- 或者是我們再設計一下實驗，例如首先我們跑了單一agent，然後到雙agent。

Slide 11：Future Work

- What have we built?
- What haven’t we finished? limitations?

1. 工具的實際建置，目前是tool call時直接回傳的fixture logs
2. 每種 artifact_type 有自己的 extractor registry。
3. 模型本身的強弱問題

Slide 12：Demo 系統(也許不一定要真的Demo)

- 如果時間來得及可demo，來不及就算了。

1. Demo 展示說明：User Prompt、Supervisor Agent: LLM/Harness和Sub Agent: LLM/Harness
2. Work Flow演示：
    - LLM一次能呼叫多個tool，由harness負責執行，執行過程中不會中途呼叫
    - Sub Agent可以視為sup agent的一項delegate工具
    - LLM 每次output有兩種：
        - Text: 但不一定會有
        - Tool use reply: 告訴harness要用什麼工具、指令要給什麼
    - Error handle: 什麼時候會回傳錯誤，錯誤怎麼處理。